Does the Order of Training Data Influence Optimization?

Mini-Project: Optimization for Machine Learning

Niels Enkaoua Teammate 1 Teammate 2

2026-01-01

Abstract

Stochastic gradient methods are usually evaluated under the assumption that mini-batches are sampled randomly. This project studies what happens when this assumption is deliberately violated. We train the same small convolutional network on Fashion-MNIST while changing only the order in which training examples are presented to SGD. Random reshuffling gives the best final test accuracy, while fixed random ordering is slightly worse. Curriculum-style orderings remain trainable but degrade generalization. More importantly, class-ordered streams produce a failure mode: with the default learning rate, the model collapses to chance-level accuracy and predicts a single class. A learning-rate diagnostic shows that reducing the step size prevents complete collapse but still leaves class-ordered training far below random reshuffling.

Introduction

Stochastic gradient descent (SGD) relies on mini-batches to provide useful noisy estimates of the full gradient. In standard supervised learning pipelines, these mini-batches are produced by randomly shuffling the training set at every epoch. This makes consecutive gradients less correlated and avoids exposing the optimizer to long non-iid segments of data. In practice, however, training data can arrive in structured orders: by class, by acquisition time, by user, by task, or by difficulty. This raises a simple empirical question: **does the order in which SGD sees the training data affect optimization and generalization?**

We answer this question by isolating data order as the main experimental variable. The dataset, model architecture, optimizer, batch size, weight decay, and random seeds are kept fixed. We compare six orderings, ranging from standard random reshuffling to pathological class-sorted streams and two simple curriculum variants. This setup is intentionally small and reproducible, but it reveals strong qualitative differences between training orders.

Experimental protocol

Dataset and model. We use Fashion-MNIST, a 10-class image classification benchmark with grayscale 28×28 clothing images (Xiao, Rasul, and Vollgraf 2017). For faster iteration, the main experiment uses a stratified subset of 20,000 training examples and evaluates on the full 10,000-example test set. The model is a compact CNN with two convolutional blocks followed by a two-layer classifier. Batch normalization and dropout are excluded so that the ordering effect is not confounded with batch-statistics or additional regularization.

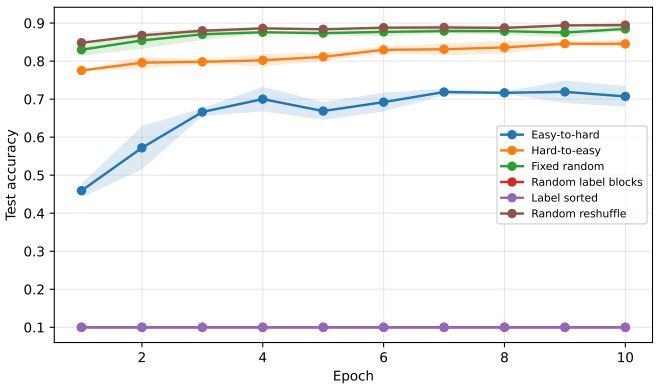
Optimizer. The main optimizer is SGD with momentum 0.9, learning rate 0.05, weight decay $5 \cdot 10^{-4}$, batch size 128, and 8 epochs. Each experiment is repeated over three seeds, and we report mean and standard deviation. The loss is cross-entropy. We also record gradient-norm statistics to diagnose instability.

Data-order strategies. The six training orders are described in Table 1. The `random` condition is the baseline used in ordinary training. The other conditions test whether removing reshuffling, presenting classes in long blocks, or ordering examples by a deterministic proxy for difficulty affects SGD.

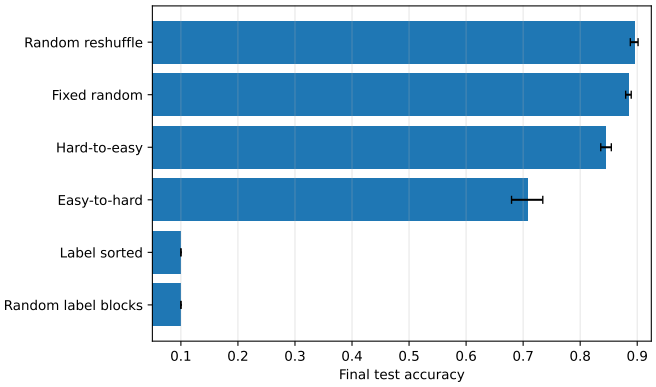
Order mode	Definition	Purpose
`random`	New random permutation at each epoch	Standard baseline
`fixed_random`	One fixed random permutation reused every epoch	Tests the value of reshuffling
`label_sorted`	All examples sorted by class label	Extreme non-iid ordering
`label_block_random`	Class blocks are shuffled, and examples are shuffled inside each block	Non-iid stream with some randomness

Order mode	Definition	Purpose
`curriculum_easy`	Examples close to their class prototype first	Simple easy-to-hard curriculum
`curriculum_hard`	Examples far from their class prototype first	Reverse curriculum

Results



(a) Test accuracy over training. The random and fixed-random runs converge fastest, while class-ordered streams fail under the default learning rate.



(a) Final test accuracy, averaged over seeds. Error bars denote standard deviation.

The main results are summarized in Table 2. Random reshuffling is the strongest baseline, reaching about 88.75% test accuracy. Reusing the same random order at every epoch is slightly worse, suggesting that epoch-wise reshuffling provides a small but measurable benefit. The curriculum orderings remain trainable, but both degrade performance; surprisingly, the hard-to-easy curriculum substantially outperforms the easy-to-hard curriculum in this setup.

Order mode	Final test accuracy, mean ± std
`random`	**88.75 ± 0.26%**
`fixed_random`	87.87 ± 0.99%
`curriculum_hard`	83.60 ± 1.65%
`curriculum_easy`	71.66 ± 0.30%
`label_sorted`	10.00 ± 0.00%
`label_block_random`	10.00 ± 0.00%

The most striking result is the complete failure of the two class-ordered streams. Since Fashion-MNIST has ten balanced test classes, chance-level accuracy is 10%. To check whether this exact value was a reporting artifact, we saved the predicted class distribution on the test set. The diagnostic showed that, at the default learning rate, `label_sorted` and `label_block_random` collapse to constant classifiers: for each seed, all 10,000 test examples are assigned to a single class. Thus the zero standard deviation is explained by the class balance of the test set, not by a metric bug.

We then ran a learning-rate diagnostic on the two pathological orderings. Lowering the learning rate from 0.05 to 0.001 prevents complete collapse, but the models remain far below the random baseline (Table 3). The prediction distributions also become more diverse, although still strongly biased toward a few classes. This suggests that class-ordered training is not merely slower; it makes SGD much more sensitive to the step size.

Order mode	Accuracy at lr = 0.05	Accuracy at lr = 0.001
`label_sorted`	10.00 ± 0.00%	21.42 ± 5.86%
`label_block_random`	10.00 ± 0.00%	43.92 ± 4.60%

Discussion

These experiments support three observations. First, the standard practice of reshuffling data is not only a convenience: it improves both stability and final performance compared with reusing a fixed order. Second, highly non-iid class-ordered streams can break SGD entirely. Consecutive mini-batches contain gradients from the same class for a long

time, so updates become strongly biased toward the current block. With momentum, this effect can accumulate and push the model toward a degenerate classifier. Third, the optimizer's sensitivity to learning rate depends strongly on the data order. A smaller learning rate rescues the class-ordered runs from complete collapse, but it does not recover the performance of random reshuffling.

The curriculum result is also informative. The easy-to-hard ordering performs much worse than the hard-to-easy ordering. One possible explanation is that our difficulty score is based on distance to class prototypes in pixel space. Images far from the class prototype may contain more diverse or boundary-relevant information, so presenting them early may help the network learn more useful features. This also shows a limitation of simple curriculum heuristics: a deterministic notion of "easy" does not necessarily produce easier optimization.

Limitations and future work

The conclusions are empirical and specific to this setup. We use one dataset, one CNN architecture, and a limited set of hyperparameters. A stronger study would repeat the experiment on CIFAR-10, compare SGD with AdamW, and tune the learning rate separately for each ordering. It would also be useful to inspect confusion matrices, per-class accuracies, and the alignment between ordered mini-batch gradients and the full gradient. Nevertheless, the observed collapse of class-ordered streams is a clear failure mode and directly illustrates how data ordering can change the behavior of stochastic optimization.

Reproducibility

The experiments can be reproduced from the repository with the following commands. The first command produces the main comparison, while the last two isolate the learning-rate diagnostic for the class-ordered streams.

```
python run.py --epochs 8 --seeds 0 1 2 --max-train-examples 20000 --force
python run.py --epochs 8 --seeds 0 1 2 --max-train-examples 20000 --orders label_sorted label_block_randon
python run.py --epochs 8 --seeds 0 1 2 --max-train-examples 20000 --orders label_sorted label_block_randon
```

All reported values are averages over three seeds. Raw epoch metrics are saved as CSV files in `results/raw/`, summary tables in `results/tables/`, and figures in `results/figures/`.

References

Xiao, Han, Kashif Rasul, and Roland Vollgraf. 2017. "Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms." *arXiv Preprint arXiv:1708.07747*.